Computer Science 2 Project
Group Progress Report
3rd Quarter

Team Members:
Therenz Ellema
Xanthus Vermug

WEEK 1 - Jan 16, 2026 (Covers week 1-3 of project schedule)

GOALS	DATA COLLECTED	PROGRESS
To define the project scope and design the initial logic flow for the "Student Time Management Program."	Researched basic Python syntax and how to structure a simple CLI application. Looked into flowcharting tools to map out the user experience	The primary achievement this week was finalizing the project concept. A logic flowchart was drafted to visualize how a student would navigate from the main menu to adding tasks. The decision was made to focus on a "Total Remaining Hours" feature as the core utility.

STRENGTH

Clear Vision: The project goals are well-defined, and the team has a solid understanding of the required features. Logical Flow: The planned navigation for the program is simple and intuitive for a student user.

AREAS FOR DEVELOPMENT

Coding Knowledge: Need to dive deeper into how Python actually handles lists and dictionaries to store the task data. No Code Yet: Currently, the project only exists as a design on paper; no actual script has been written.

STRATEGY FOR SUCCESS

Documentation First: Keeping the design documents updated so that when coding starts, there is a clear roadmap to follow. Resource Gathering: Finding reliable Python tutorials specifically for data storage and menu loops to prepare for Week 2.

Computer Science 2 Project
Group Progress Report
3rd Quarter

Team Members:
Therenz Ellema
Xanthus Vermug

WEEK 2 - Jan 23, 2026 (Covers week 4 of project schedule)

GOALS	DATA COLLECTED	PROGRESS
To transform the Week 1 design into actual Python code. To set up the list structures to hold task data and create the main menu functionality.	Researched Python list and dictionary methods for storing data. Collected logic for while loops to keep the program running and input() functions for user data.	We successfully coded the "skeleton" of the program. We created the main menu where users can choose to add or view tasks. We implemented the math logic for the "Total Remaining Hours," ensuring the program correctly adds up the numbers entered. The basic "Add Task" feature is now fully functional.

STRENGTH

Accurate Logic: The core math for calculating remaining time works perfectly. Stable Framework: The main loop is solid, so the user can move between adding and viewing tasks without the program closing unexpectedly.

AREAS FOR DEVELOPMENT

Messy display: the data just prints as a raw list; it doesn't look like an "organized table" yet.
Input Errors: If a user types a letter instead of a number for "Hours," the program crashes.

STRATEGY FOR SUCCESS

Logic First: We are focusing on building a "bug-free" foundation for the task input system and the time calculation math. We believe it is better to have a plain-looking program that works perfectly than a pretty program that crashes.

Step-by-Step Coding: Instead of writing the whole code at once, we are coding and testing one feature at a time (e.g., making sure "Add Task" works before starting on "View Tasks").

Team Collaboration: We are double checking each other's code to catch typos or logic errors early. This keeps our development phase efficient and ensures that the "Total Remaining Hours" formula is 100% accurate before we move on to the final UI design in Week 3.

Computer Science 2 Project
Group Progress Report
3rd Quarter

Team Members:
Therenz Ellema
Xanthus Vermug

WEEK 3 - Jan 30, 2026 (Covers week 5 of project schedule)

GOALS	DATA COLLECTED	PROGRESS
To clean up the interface for a professional look. To add error handling (try-except) and ensure the "Task Status" (Pending/Completed) updates correctly.	Collected data on f-string formatting to create aligned columns. Researched Conditional Statements to allow users to toggle tasks from "Pending" to "Completed."	We finished the project this week. We replaced the messy list output with a clean, structured table that aligns the Name, Priority, and Status. We also added "input validation" so the program no longer crashes if a user makes a typo. The "Task Status" feature is now interactive, allowing users to check off tasks as they finish them.

STRENGTH

Professional UI: The table layout is very easy to read and looks like a real application.

User-Proof: We added error handling, making the program much more reliable for the average student.

Lightweight: The final code runs instantly and is very efficient

AREAS FOR DEVELOPMENT

Temporary Storage: Since we didn't use external files (like .txt), the data is lost when you close the program.

Manual Sorting: Tasks stay in the order they were typed; it doesn't automatically move High Priority tasks to the top yet.

STRATEGY FOR SUCCESS

To ensure the project's success, we focused on making the code "unbreakable." We spent the final days testing every possible input to make sure a student wouldn't get frustrated by bugs. We also prioritized the visual table layout because we realized in Week 2 that a program is only useful if the information is easy to read at a glance.

Computer Science 2 Project
Group Progress Report
3rd Quarter

Team Members:
Therenz Ellema
Xanthus Vermug

WEEK 4 – Feb 6, 2026 (Covers week 6 of project schedule)

GOALS	DATA COLLECTED	PROGRESS
To finalize the program by adding a save/load feature and performing a final "bug hunt" before completion.	Researched basic Python syntax and how to structure a simple CLI application. Looked into flowcharting tools to map out the user experience	The program is now essentially complete. The "Task Status" and "Automated Sorting" features from last week were polished, and a basic save-to-file function was added. Extensive testing was done to ensure the input validation (try-except blocks) catches all potential user errors.

STRENGTH

User Reliability: The program is now very stable; it handles "bad" inputs gracefully without crashing and saves data between sessions. Professional Look: The organized table format for displaying tasks makes the data easy to read at a glance.

AREAS FOR DEVELOPMENT

Code Cleanup: The script has grown quite large, so some sections could benefit from better commenting or being broken into more functions for readability. Feature Creep: There are ideas for more complex features (like notifications), but these might be out of scope for the current deadline.

STRATEGY FOR SUCCESS

Final Testing: Running the program through a "stress test" by having someone outside the team try to break it by entering weird data.

Polishing Comments: Ensuring every block of code is explained clearly so the teacher can easily follow the logic during the final submission.